

Task 3: Evaluation

- When comparing the different prediction strategies, you must first report their **key parameters** and corresponding **hardware costs** (simplified as memory footprint).
 - You are required to compare the **performance metrics** (e.g., accuracy, control hazards) among the baseline (no prediction) and each implemented strategy. You should use some or all of the provided test cases. If you find the existing test cases insufficient, you are encouraged to write your own and document their usage in your report. Your comparison and analysis of the different strategies must be presented in your report.
-

4. Report Requirements

Your lab report must be written in English and submitted in PDF format. It should include:

- A detailed description of your design and implementation of the branch predictors and the pipeline integration mechanism.

implementation of the branch predictors:

1. Always Not Taken (NT)

```
bool NT_strategy() {return false;};  
bool AT_strategy() {return true;};  
BR_STRATEGY = NT;
```

2. Always Taken (AT)

```
bool NT_strategy() {return false;};  
bool AT_strategy() {return true;};  
BR_STRATEGY = AT;
```

3. One-bit Predictor (1-Bit)

```

class OneBitBR{
public:
    OneBitBR():history_(false) {};
    bool predict() const {
        return history_;
    }
    void update(bool taken) {
        history_ = taken;
    }
private:
    bool history_;
};

```

4. Two-bit Predictor (2-Bit)

```

class BHT2Bit {
public:
    BHT2Bit(size_t K=20) : K(K) {
        size_t words = (K + 15) / 16;
        table.resize(words, 0xAAAAAAAA);
    }

    bool predict(uint64_t pc) const {
        size_t idx = (pc/4) % K;
        uint8_t state = get_state(idx);
        return state ≥ 2; // 2 = Weakly Taken, 3 = Strongly Taken
    }
    // 更新
    void update(uint64_t pc, bool taken) {
        size_t idx = (pc/4) % K;
        uint8_t state = get_state(idx);
        if (taken) {
            if (state < 3) state++;
        } else {
            if (state > 0) state--;
        }
        set_state(idx, state);
    }

private:
    size_t K;
    std::vector<uint32_t> table; // 每个 uint32_t 存 16 个 entry

```

5. Perceptron Predictor

```
class PercptBR{
public:
    PercptBR(size_t history_sz=60,size_t table_sz=70,uint32_t thres=8)
        : history_sz_(history_sz) ,
          table_sz_(table_sz),
          threshold_(thres) ,
          history_(0),
          y_last_(0),
          wtable_()
    {
        wtable_.resize(table_sz_, std::vector<int32_t>(history_sz_+1, 0));
    }

private:
    size_t history_sz_;
    size_t table_sz_;
    uint32_t threshold_;
    uint64_t history_; //前一个周期的执行情况表1,2,3
    int32_t y_last_;
    std::vector<std::vector<int32_t>> wtable_; // 每个 perception12-62个权重 w0表示初始参数
};
```

```

bool predict(uint64_t pc) {
    size_t idx = (pc/4) % table_sz_;
    int32_t y_out=wtable_[idx][history_sz_];
    for(size_t i=0;i<history_sz_;i++){ //0位是最新分支
        int32_t h = ((history_ >> i) & 0x1) ? 1 : -1;
        y_out += wtable_[idx][i] * h;
    }
    y_last_=y_out;
    // // 打印调试信息
    // std::cout << "[Predict] PC: " << pc
    //          << ", idx: " << idx
    //          << ", y_out: " << y_out
    //          << ", history: 0x" << std::hex << history_ << std::dec
    //          << ", prediction: " << (y_out>0 ? "Taken" : "Not Taken")
    //          << std::endl;
    return y_out>0;
}

```

// 更新

```

void update(uint64_t pc, bool taken) {
    bool predicted= (y_last_>0);
    size_t idx = (pc/4) % table_sz_;
    int32_t param= (taken ? 1 : -1);
    wtable_[idx][history_sz_]= wtable_[idx][history_sz_]+param;
    if(taken!=predicted // abs(y_last_)<threshold_){
        for(size_t i=0;i<history_sz_;i++){
            int32_t h = ((history_ >> i) & 0x1) ? 1 : -1;
            wtable_[idx][i]= wtable_[idx][i]+param*h;
        }
    }
    history_ = ((history_ << 1) / (taken ? 1 : 0)) & ((1ULL << history_sz_) - 1);
    // // 打印更新信息
    // std::cout << "[Update] PC: " << pc
    //          << ", taken: " << (taken ? "T" : "N")
    //          << ", predicted: " << (predicted ? "T" : "N")
    //          << ", new history: 0x" << std::hex << history_ << std::dec
    //          << ", w0: " << wtable_[idx][history_sz_] << std::endl;
}

```

6. Universal interface

```

class BR_predict{
public:
    //跳转地址的计算在deocode阶段由模拟器完成
    //分支预测只决定是否跳转
    BR_predict(const Options& opts)
        :strategy_(opts.branch_predict_policy)
    {
        OB_ = std::make_unique<OneBitBR>();
        TB_ = std::make_unique<BHT2Bit>(opts.K);
        PB_ = std::make_unique<PercptBR>(opts.history_sz, opts.table_sz,
opts.threshold);
    }
};

```



```

bool strategy(uint64_t pc){
    switch(strategy_){
        case BR_STRATEGY::NT:
            return NT_strategy();
        case BR_STRATEGY::AT:
            return AT_strategy();
        case BR_STRATEGY::OB:
            return OB_->predict();
        case BR_STRATEGY::TB:
            return TB_->predict(pc);
        case BR_STRATEGY::PERCPT:
            return PB_->predict(pc);
        default:
            return NT_strategy();
    }
};

void UpdateStrategy(uint64_t pc, bool taken){
    switch(strategy_){
        case BR_STRATEGY::OB:
            OB_->update(taken);
            break;
        case BR_STRATEGY::TB:
            TB_->update(pc, taken);
            break;
        case BR_STRATEGY::PERCPT:
            PB_->update(pc, taken);
            break;
        default:
            break;
    }
}

private:
    bool NT_strategy() {return false;};
    bool AT_strategy() {return true;};
    BR_STRATEGY strategy_;
    //1-bit saturating counter
    std::unique_ptr<OneBitBR> OB_ = nullptr;
    //2-bit
    std::unique_ptr<BHT2Bit> TB_ = nullptr;
    //
    std::unique_ptr<PercptBR> PB_ = nullptr;

};

```

pipeline integration mechanism:

1. support in data structure

```
struct PipeOp {  
    RegId rs1 = -1, rs2 = -1;  
    int64_t op1 = 0, op2 = 0;  
    RegId dest_reg = -1;  
    int64_t offset = 0;  
    // internal decode result  
    std::string inst_str;  
  
    // execute  
    int64_t out = 0;  
    bool write_mem = false;  
    bool read_mem = false;  
    bool read_sign_ext = false;  
    uint32_t mem_len = 0;  
    bool branch = false;  
    uint32_t jump_pc = 0;  
    //branch_predictor-adding  
    bool branch_taken = false; //默认情况: always not taken  
    uint32_t branch_target = 0;  
};
```

2. predict address calculated in deocode stage

```
void DecodeInst(PipeOp* op, const Regs& regs) {
    switch (opcode) {
        case OP_JAL:
            op1 = imm_uj;
            op2 = 0;
            offset = imm_uj;
            dest_reg = rd;
            inst_type = JAL;
            inst_name = INSTNAME[inst_type];
            op1_str = std::to_string(imm_uj);
            dest_str = REGNAME[dest_reg];
            inst_str = inst_name + " " + dest_str + "," + op1_str;
            branch_target = pc + op1;
            break;
        case OP_JALR:
            op1 = regs[rs1];
            reg1 = rs1;
            op2 = imm_i;
            dest_reg = rd;
            inst_type = JALR;
            inst_name = INSTNAME[inst_type];
            op1_str = REGNAME[rs1];
            op2_str = std::to_string(op2);
            dest_str = REGNAME[dest_reg];
            inst_str = inst_name + " " + dest_str + "," + op1_str + "," + op2_str;
            branch_target = (op1 + op2) & (~(uint64_t)1);
    }
}
```

```

void DecodeInst(PipeOp* op, const Regs& regs) {
    switch (opcode) {
        break;
    case OP_JALR:
        op1 = regs[rs1];
        reg1 = rs1;
        op2 = imm_i;
        dest_reg = rd;
        inst_type = JALR;
        inst_name = INSTNAME[inst_type];
        op1_str = REGNAME[rs1];
        op2_str = std::to_string(op2);
        dest_str = REGNAME[dest_reg];
        inst_str = inst_name + " " + dest_str + "," + op1_str + ","
        branch_target = (op1 + op2) & (~(uint64_t)1);
        break;
    case OP_BRANCH:
        op1 = regs[rs1];
        op2 = regs[rs2];
        reg1 = rs1;
        reg2 = rs2;
        offset = imm_sb;
        branch_target = pc + offset;
    }
}

```

3. mispredicted handling

```

void FiveStageSimulator::Fetch() {
    //分支预测错误 不需要进行排空, 因为下一个周期就会进行恢复
    if(true == mispredicted_branch){
        if(verbose_) {
            printf(" Branch misprediction recovery at fetch\n");
        }
        return;
    }
    // control hazard 1. 分支预测taken后的等待
    if (true == wait_for_branch_) {
        if(verbose_) {
            printf("control hazard at fetch\n");
        }
        // debug information
        history_.control_hazard_count++;
        return;
    }
}

```



```

void FiveStageSimulator::Decode() {
    /* if no op to decode, return */
    if (decode_op_ == nullptr) {
        if (verbose_) {
            printf("decode: Bubble\n");
        }
        return;
    }
    //分支预测错误
    if(true == mispredicted_branch_){
        if (verbose_) {
            printf(" Branch misprediction recovery at decode\n");
        }
        //清除流水线中的内容
        decode_op_ = nullptr;
        return;
    }
    auto* op = decode_op_.get();
    if (op->pc_len != 4) {
        Panic(
            "Current implementation does not support 16bit RV64C instructions!\n");
    }
}

```

4. predicting logic in decode stage

```

void FiveStageSimulator::Decode() {
    if(enable_predict){
        // control hazard (wait_for_branch_用于暂停本周期的取指)
        //分支预测逻辑
        bool& branch_taken = op->branch_taken;
        if(IsBranch(op->inst_type) || IsJump(op->inst_type)){ //如果是分支指令，则进行分支预测
            if(predictor->strategy(op->pc)){
                branch_taken=true;
                PipeRecover(op->branch_target); //在这里进行了预测为taken的跳转，流水线延迟一个时钟周期
                if (verbose_) {
                    std::cout << fmt::format(
                        " Branch prediction: taken\n");
                }
                wait_for_branch_=true;
            }else{
                branch_taken=false;
                wait_for_branch_=false; //继续进行下一条指令的取指+4
                if (verbose_) {
                    std::cout << fmt::format(
                        " Branch prediction: not taken\n");
                }
            }
            history_.predicted_branch_count++; //预测次数+1
        }
    }
    else wait_for_branch_ = IsBranch(op->inst_type) || IsJump(op->inst_type);
    /* place op in downstream slot */
}

```

5. predict verification in execute stage

```

void FiveStageSimulator::Execute() {
if(enable_predict){
    if (IsBranch(op->inst_type) // IsJump(op->inst_type)) {
        //这里在分支预测正确的情况下，不进行任何操作
        //分支加上分支标志
        if(op->branch_taken){ //预测taken
            if(op->jump_pc != op->branch_target){ //预测错误
                if (verbose_) {
                    std::cout << fmt::format(
                        " Branch misprediction: predicted taken but actually not taken\n");
                }
                predictor_>UpdateStrategy(op->pc,false); //更新为not taken
                PipeRecover(op->jump_pc); //恢复正确的pc
                mispredicted_branch_=true;
            }else{
                predictor_>UpdateStrategy(op->pc,true); //预测正确，继续保持
                history_.correct_branch_count++;
            }
        }else{ //预测not taken
            if(op->jump_pc == op->branch_target){ //预测错误
                if (verbose_) {
                    std::cout << fmt::format(
                        " Branch misprediction: predicted taken but actually not taken\n");
                }
                predictor_>UpdateStrategy(op->pc,true); //更新为taken
                PipeRecover(op->jump_pc); //恢复正确的pc
                mispredicted_branch_=true;
            }else{
                predictor_>UpdateStrategy(op->pc,false); //预测正确，继续保持
                history_.correct_branch_count++;
            }
        }
    }
}
}

```

- A comprehensive evaluation comparing the predictors, covering their parameter settings, hardware costs, and measured performance, along with an analysis explaining the observed differences..

test_branch程序

1. Always Not Taken (NT)

```

./build/simulator -i test/build/with-syscall/test_branch.riscv --
branch_predict_policy "Always Not Taken (NT)"
Yes, f2 is true
a[5] = 1 2 3 4 5
a[5] = 1 12 123 1234 12345
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 1159
Number of Cycles: 2703
Avg Cycles per Instrcuton: 2.3322
Number of Control Hazards: 368
Number of Data Hazards: 1175
Branch Prediction Accuracy: 6.12% (12 / 196)

```

2. Always Taken (AT)

```
./build/simulator -i test/build/with-syscall/test_branch.riscv --
branch_predict_policy "Always Taken (AT)"
Yes, f2 is true
a[5] = 1 2 3 4 5
a[5] = 1 12 123 1234 12345
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 1159
Number of Cycles: 2545
Avg Cycles per Instrcution: 2.1959
Number of Control Hazards: 208
Number of Data Hazards: 1177
Branch Prediction Accuracy: 93.88% (184 / 196)
```

3. One-bit Predictor (1-Bit)

```
./build/simulator -i test/build/with-syscall/test_branch.riscv --
branch_predict_policy "One-bit Predictor (1-Bit)"
Yes, f2 is true
a[5] = 1 2 3 4 5
a[5] = 1 12 123 1234 12345
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 1159
Number of Cycles: 2556
Avg Cycles per Instrcution: 2.2053
Number of Control Hazards: 221
Number of Data Hazards: 1175
Branch Prediction Accuracy: 87.24% (171 / 196)
```

4. Two-bit Predictor (2-Bit)

```
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 1159
Number of Cycles: 2548
Avg Cycles per Instrcution: 2.1984
Number of Control Hazards: 211
Number of Data Hazards: 1177
Branch Prediction Accuracy: 91.84% (180 / 196)
```

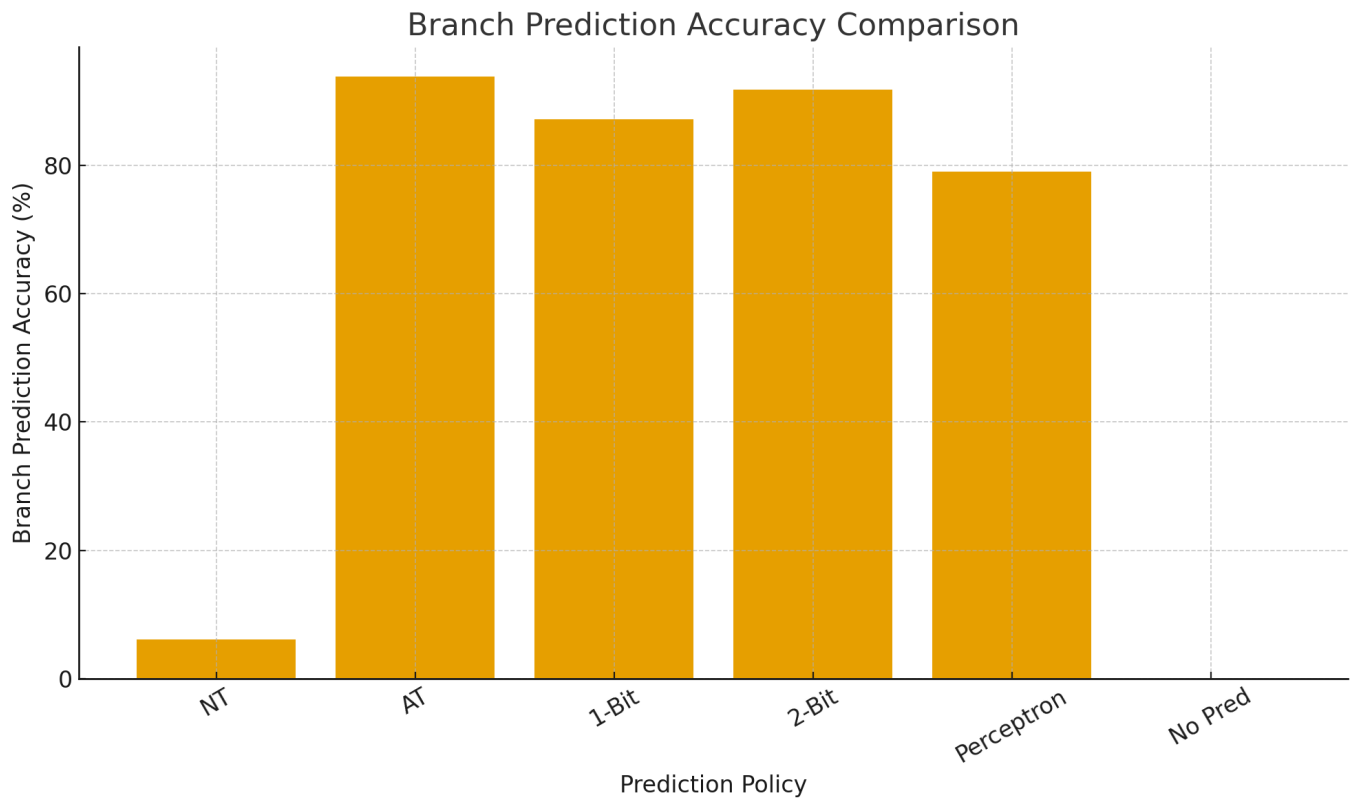
5. Perceptron Predictor

```
Program exit from an exit() system call
----- STATISTICS -----
```

```
Number of Instructions: 1159
Number of Cycles: 2566
Avg Cycles per Instrcution: 2.2140
Number of Control Hazards: 231
Number of Data Hazards: 1175
Branch Prediction Accuracy: 79.08% (155 / 196)
```

6. NO prediction

```
./build/simulator -i test/build/with-syscall/test_branch.riscv
Yes, f2 is true
a[5] = 1 2 3 4 5
a[5] = 1 12 123 1234 12345
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 1159
Number of Cycles: 2727
Avg Cycles per Instrcution: 2.3529
Number of Control Hazards: 392
Number of Data Hazards: 1175
Branch Prediction Accuracy: 0.00% (0 / 0)
```



Strategy	Accuracy	Performance / Notes
Always Not Taken (NT)	6.12%	Essentially no predictive power; only works in programs where branches are almost never taken.

Strategy	Accuracy	Performance / Notes
Always Taken (AT)	93.88%	Performs very well here because most branches in this test program are taken.
One-bit Predictor (1-Bit)	87.24%	Learns the last branch behavior, but can be affected by loop-boundary oscillations.
Two-bit Predictor (2-Bit)	91.84%	Adds hysteresis to reduce misprediction on loop-boundary oscillations, more robust than 1-bit.
Perceptron Predictor	79.08%	Can learn long histories, but in this test program branch correlations are weak, so advantages are not fully realized.
No Prediction	0%	Makes no prediction; degenerates to always stalling.

Ackermann程序测试

1. Always Not Taken (NT)

```
./build/simulator -i test/build/with-syscall/test_branch.riscv --
branch_predict_policy "Always Not Taken (NT)"
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 431309
Number of Cycles: 978555
Avg Cycles per Instruction: 2.2688
Number of Control Hazards: 96210
Number of Data Hazards: 451035
Branch Prediction Accuracy: 22.35% (13847 / 61952)
```

2. Always Taken (AT)

```
./build/simulator -i test/build/with-syscall/ackermann.riscv -p --
branch_predict_policy "Always Taken (AT)"
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 431309
Number of Cycles: 965073
Avg Cycles per Instruction: 2.2375
Number of Control Hazards: 75799
Number of Data Hazards: 457964
Branch Prediction Accuracy: 77.65% (48105 / 61952)
```

3. One-bit Predictor (1-Bit)

```
./build/simulator -i test/build/with-syscall/ackermann.riscv -p --
branch_predict_policy "One-bit Predictor (1-Bit)"
----- STATISTICS -----
Number of Instructions: 431309
Number of Cycles: 971503
Avg Cycles per Instruction: 2.2525
Number of Control Hazards: 88915
Number of Data Hazards: 451278
Branch Prediction Accuracy: 56.08% (34745 / 61952)
```

4. Two-bit Predictor (2-Bit)

```
./build/simulator -i test/build/with-syscall/ackermann.riscv -p --
branch_predict_policy "Two-bit Predictor (2-Bit)"
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 431309
Number of Cycles: 939227
Avg Cycles per Instruction: 2.1776
Number of Control Hazards: 49954
Number of Data Hazards: 457963
Branch Prediction Accuracy: 98.15% (60805 / 61952)
```

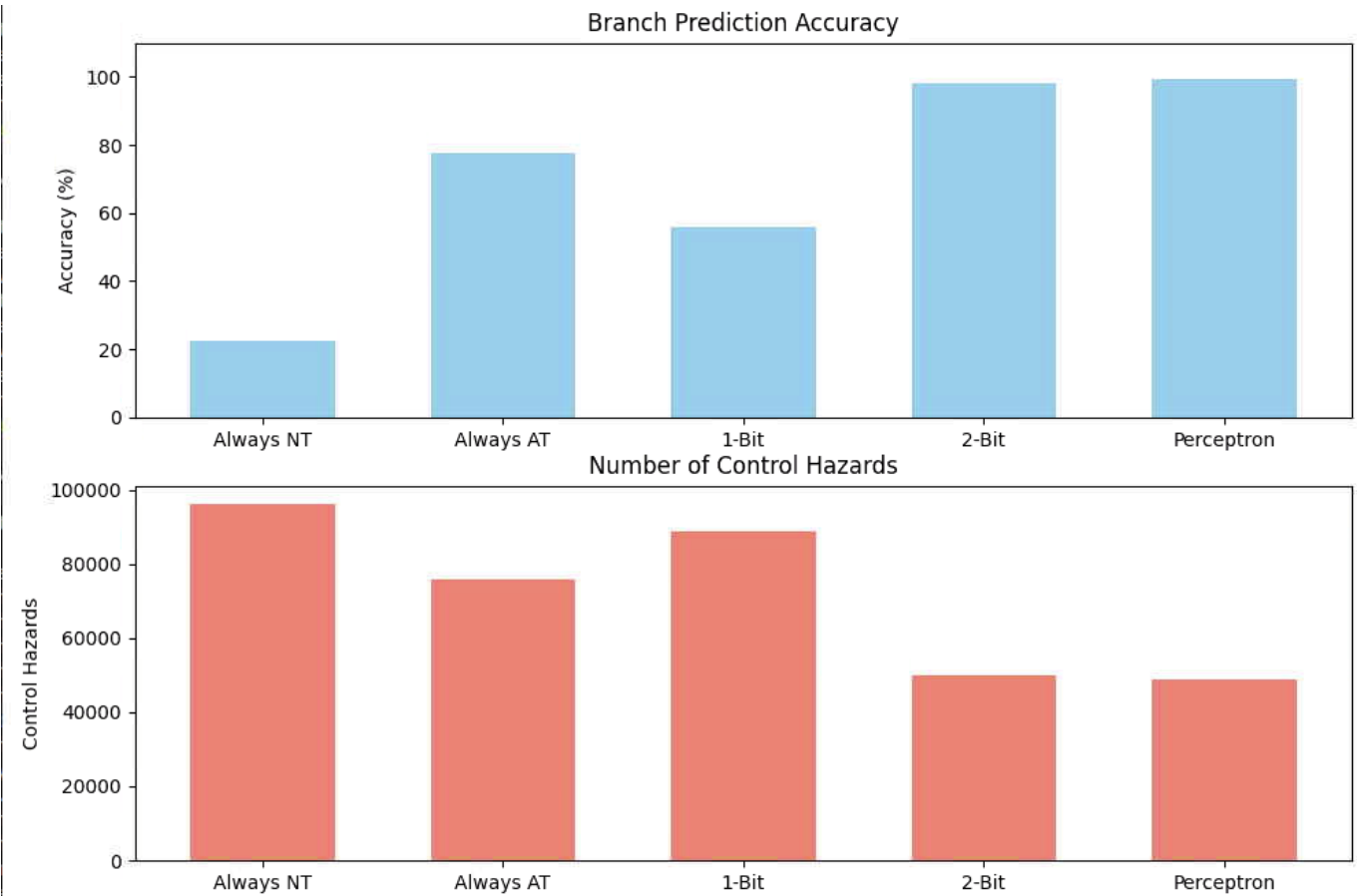
5. Perceptron Predictor

```
./build/simulator -i test/build/with-syscall/ackermann.riscv -p --
branch_predict_policy "Perceptron Predictor"
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 431309
Number of Cycles: 938120
Avg Cycles per Instruction: 2.1751
Number of Control Hazards: 48848
Number of Data Hazards: 457962
Branch Prediction Accuracy: 99.24% (61483 / 61952)
```

6. NO prediction

```
./build/simulator -i test/build/with-syscall/ackermann.riscv
----- STATISTICS -----
Number of Instructions: 431309
Number of Cycles: 1006249
Avg Cycles per Instruction: 2.3330
Number of Control Hazards: 123904
Number of Data Hazards: 451035
```

Branch Prediction Accuracy: 0.00% (0 / 0)



Predictor	Prediction Accuracy	Explanation
Always Not Taken (NT)	22.35%	Always predicts branches as not taken. Most branches in the Ackermann function are actually taken due to deep recursion, so the accuracy is very low and control hazards are high.
Always Taken (AT)	77.65%	Always predicts branches as taken. Most branches are taken, especially in the recursive cases, so accuracy is much higher than NT. Only a few branches at boundary conditions (<code>m == 0</code> or <code>n == 0</code>) are mispredicted.
One-bit Predictor	56.08%	Uses a single global bit per branch. In the Ackermann function, some branches alternate between taken and not taken frequently, leading to frequent mispredictions and moderate accuracy.
Two-bit Predictor	98.15%	Uses a 2-bit saturating counter per branch, which tolerates occasional mispredictions. This works well for the highly regular taken branches in the Ackermann recursion, resulting in very high accuracy and fewer control hazards.
Perceptron Predictor	99.24%	Uses global branch history and learned correlations between branches. Captures complex patterns in recursive calls, achieving almost perfect prediction accuracy and the lowest number of control hazards.

opts

1. use -p option to enable prediction

```
app.add_flag("-p,--enable_predict", opts.enable_predict, "Enable predict");
```

2. prediction strategy

```
std::string branch_predict_policy_str = "Always Not Taken (NT)";
std::map<std::string, BR_STRATEGY> branch_predict_map = {
    {"Always Not Taken (NT)", BR_STRATEGY::NT},
    {"Always Taken (AT)", BR_STRATEGY::AT},
    {"One-bit Predictor (1-Bit)", BR_STRATEGY::OB},
    {"Two-bit Predictor (2-Bit)", BR_STRATEGY::TB},
    {"Perceptron Predictor", BR_STRATEGY::PERCPT}
};

app.add_option("--branch_predict_policy", branch_predict_policy_str,
    "branch_predict policy: NT, or AT, or 1-Bit, or 2-Bit, or Perceptron")
    →check(CLI::IsMember({"Always Not Taken (NT)", "Always Taken (AT)",
        "One-bit Predictor (1-Bit)", "Two-bit Predictor (2-Bit)", "Perceptron Predictor"}))
    →default_val("Always Not Taken (NT)");
```

3. Perceptron Predictor parameter set

```
app.add_option("--history_sz", opts.history_sz, "PercptBR:History register")
    →default_val(opts.history_sz);
app.add_option("--table_sz", opts.table_sz, "PercptBR:Table of Perceptions")
    →default_val(opts.table_sz);
app.add_option("--threshold", opts.threshold, "PercptBR:threshold of Weight")
    →default_val(opts.threshold);
```

```
class PercptBR{
public:
    PercptBR(size_t history_sz=60,size_t table_sz=70,uint32_t thres=8)
        : history_sz_(history_sz) ,
          table_sz_(table_sz),
          threshold_(thres) ,

```

4. Two-bit Predictor (2-Bit) parameter set

```
app.add_option("--K", opts.K, "BHT2Bit:BHT entry nums")
    →default_val(opts.K);
```

```
class BHT2Bit {
public:
    BHT2Bit(size_t K=20) : K(K) {
        size_t words = (K + 15) / 16;
        table.resize(words, 0xAAAAAAAA);
    }

```

example: ./build/simulator -i test/build/with-syscall/ackermann.riscv -p --branch_predict_policy "Perceptron Predictor"